

Challenge C8：微信群 → 班级管理子系统（WeChat Group as Class Management Agent Skill）

🔥 把一个微信群变成一个 **Agent** 可调用的班级管理系统——不是替换微信，而是让 Agent 能 **读懂、追踪、行动** 群里发生的一切。

本质：WeChat Group 是一个 **Channel**（P3394 术语），班级管理是一个 **Skill Domain**，Agent Runtime（OpenClaw / Claude Code）是 **执行层**。你的任务是把这三层连起来。

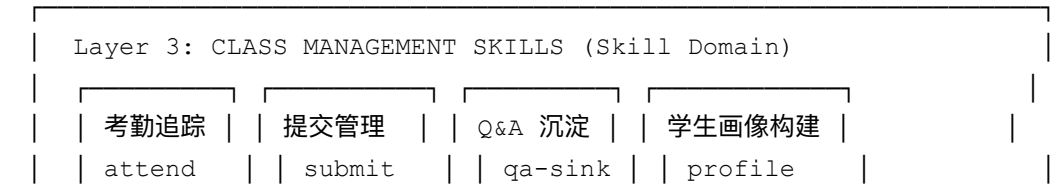
🧠 为什么做这个

Elite 20 课程目前的管理现实：

问题	现状	理想状态
提交追踪	老师手动翻群文件	Agent 自动识别并归档
考勤/活跃度	全凭感觉	每个人的发言频率、质量、时间线可查
问答沉淀	好问题淹没在聊天流里	自动提取 Q&A 对，建立课程 FAQ
通知确认	发了通知不知道谁看了	Agent 追踪已读/已回复/已行动
学生画像	不了解每个学生的强项和困难	从群互动中自动构建学生能力画像

核心洞察：微信群已经是中国大学课程管理的事实标准——但它只是一个 **消息流**。我们不需要替换它，只需要在它上面加一层 **Agent 智能层**，让消息流变成可查询、可追踪、可行动的结构化数据。

🏗️ 架构概览：三层蛋糕





类比：把这想象成给一栋已有的大楼（微信群）装一套智能楼宇管理系统。你不拆楼，你在楼的管线上装传感器（Layer 1），连到控制中心（Layer 2），然后运行各种管理程序（Layer 3）。

领域模型（Domain Model First）

在写任何代码之前，先建立班级管理的领域模型。这是一个 **Property Graph**：

节点类型（Node Types）

```
Student:
  properties:
    name: string
    wechat_id: string          # 微信群内昵称/ID
```

```

    major: string
    enrollment_date: date
    skill_tags: [string]          # 从群互动中推断

Challenge:
  properties:
    id: string                    # C1, C2A, C3C, C4B...
    name_cn: string
    name_en: string
    due_date: date?
    levels: [Bronze, Silver, Gold, Platinum]

Submission:
  properties:
    timestamp: datetime
    files: [{name, type, size, path}]
    level_claimed: string
    status: enum[received, reviewing, accepted, needs_revision]

Message:
  properties:
    timestamp: datetime
    content: string
    type: enum[text, image, file, voice, link, system]
    wechat_msg_id: string

QAPair:
  properties:
    question: string
    answer: string
    topic_tags: [string]
    quality_score: float          # 0-1

Report:
  properties:
    type: enum[weekly, monthly, challenge_summary]
    generated_at: datetime
    data_snapshot: JSON

```

边类型 (Edge Types)

SUBMITTED:	Student → Submission → Challenge	# 谁提交了什么挑战
SENT:	Student → Message	# 谁发了什么消息
ASKED:	Student → QAPair	# 谁问了什么
ANSWERED:	Student → QAPair	# 谁回答了什么
MENTORED:	Student → Student	# 谁帮了谁 (从回复关系推断)

PROGRESSED_TO: Student → Challenge.Level

当前进度

TAGGED_WITH: Message → Challenge

消息属于哪个挑战的讨论

KSTAR 记忆编码

每条有意义的群交互都编码为一个 KSTAR Episode:

K: 课程上下文 + 已知的学生画像 + 挑战定义

S: {who: Student, what: MessageType, where: GroupChat, when: timestamp,
context: recent_discussion_thread}

T: {classify_and_route | extract_submission | build_qa | update_profile}

A: Agent 的实际操作 (分类、存储、回复、标记)

R: 操作结果 + 更新后的状态

四级任务

Level 1: 数据桥接——让 Agent 能“看到”群消息 (Bronze 🏅)

目标: 从本地微信数据中提取一个群的消息流, 输出为 Agent 可消费的 JSON。

任务清单

1. 选择并安装 wechat-cli 工具

- 推荐起点: freestylefly/wechat-cli (Python, CLI-first, JSON 输出)
- 备选: sjzar/chatlog (HTTP API + TUI) 、 afumu/wetrace (取证级)
- 如果以上都不适用你的平台, 可用任何能从本地微信数据库提取消息的工具

2. 提取目标群的消息流

- 指定群名称或 ID
- 提取最近 N 天的消息 (建议从 7 天开始)
- 输出格式: JSON Lines (每行一条消息)

3. 消息标准化

- 将原始消息转换为标准 schema:

```
{  
  "msg_id": "unique_id",
```

```
"timestamp": "2026-04-10T14:30:00+08:00",
"sender": {"wechat_id": "xxx", "nickname": "张三"},
"type": "text|image|file|voice|link|system",
"content": "消息文本内容",
"media_ref": "path/to/attachment_if_any",
"reply_to": "parent_msg_id_if_reply"
}
```

4. 写一个 CLI wrapper

- 封装成一个可调用的命令：

```
python wechat_bridge.py --group "Elite20" --days 7 --output messages.jsonl
```

交付物

- wechat_bridge.py — CLI 桥接脚本
- sample_output.jsonl — 示例输出（可脱敏）
- SETUP.md — 安装和配置步骤
- 姓名_C8_L1_AI日志.md — 你与 AI 的协作过程

验收标准

- 能从本地微信数据中成功提取至少 100 条群消息
- 输出符合上述 JSON schema
- 能区分文本、文件、图片等消息类型
- CLI 可重复运行，支持时间范围参数

Level 2: 消息分类与信号提取 (Silver)

目标：让 Agent 不只能“看到”消息，还能“理解”每条消息的意图——是提交？是提问？是讨论？是闲聊？

任务清单

1. 消息分类器 (Message Classifier)

- 为每条消息打上意图标签：

标签	含义	信号特征
submission	挑战提交	包含文件 + 匹配挑战命名规范
question	提问	问号、“怎么”、“为什么”、“请问”
answer	回答	回复一个 question 消息
discussion	讨论	围绕某话题的多条消息
announcement	通知/公告	老师发的、@全体、包含deadline
resource	分享资源	链接、文件、推荐
social	日常闲聊	不属于以上任何一类

- 实现方式（任选）：
 - 规则引擎：基于关键词 + 正则 + 消息结构（Bronze 级）
 - LLM 分类：用本地 LLM 或 API 做意图分类（Silver 级）
 - 混合：规则预筛 + LLM 精分类（Gold 级）

2. 挑战关联器（Challenge Linker）

- 将消息关联到具体挑战（C1-C8）
- 使用 `signal_keywords` 和 `signal_extensions` 匹配
- 输出：每条消息增加 `challenge_ref` 字段

3. 提交检测器（Submission Detector）

- 当检测到文件上传 + 命名规范匹配时，自动创建 Submission 记录
- 检查完整性：AI日志是否提交？必要附件是否齐全？
- 输出： `submissions.json` （按学生×挑战的矩阵）

4. Q&A 提取器（QA Extractor）

- 从消息流中提取问答对
- 匹配 question → answer 关系（基于回复链或时间窗口）
- 输出： `qa_pairs.json`

交付物

- classifier.py — 消息分类器
- challenge_linker.py — 挑战关联器
- submission_detector.py — 提交检测器
- qa_extractor.py — Q&A 提取器
- 测试数据 + 分类结果样例
- 姓名_C8_L2_AI日志.md

验收标准

- 分类器准确率 > 70% (在标注的测试集上)
- 能正确检测出至少 5 个提交事件
- 能提取出至少 10 个有意义的 Q&A 对
- 所有模块可独立运行，也可 pipeline 组合

Level 3: Agent Skill 封装——让任何 Agent 都能调用 (Gold 🏆)

目标：把 Level 1-2 的功能封装为标准 Agent Skill，可被 OpenClaw / Claude Code / 任何 P3394 兼容 Agent 调用。

架构要求

每个 Skill 遵循标准 SKILL.md 格式：

```
wechat-class-manager/
├── SKILL.md                # 技能说明（触发词、IO、依赖）
├── skills/
│   ├── wechat-bridge/     # Layer 1: 数据桥接
│   │   ├── SKILL.md
│   │   └── bridge.py
│   ├── msg-classifier/    # Layer 2: 消息分类
│   │   ├── SKILL.md
│   │   └── classifier.py
│   ├── submission-tracker/ # Layer 3: 提交追踪
│   │   ├── SKILL.md
│   │   └── tracker.py
│   ├── qa-sink/           # Layer 3: Q&A 沉淀
│   │   └── SKILL.md
```

```

|   |   └─ sink.py
|   └─ student-profile/          # Layer 3: 学生画像
|   |   └─ SKILL.md
|   |   └─ profiler.py
|   └─ class-report/            # Layer 3: 报告生成
|   |   └─ SKILL.md
|   |   └─ reporter.py
└─ schemas/
    └─ message.schema.json      # 消息 schema
    └─ submission.schema.json   # 提交 schema
    └─ student.schema.json      # 学生画像 schema
    └─ domain-graph.yaml        # Property Graph schema
└─ config/
    └─ challenges.yaml          # 挑战定义 (来自课程配置)
    └─ group_config.yaml        # 群配置 (群名、成员映射等)
└─ tests/
    └─ test_classifier.py
    └─ test_submission.py
    └─ fixtures/                # 测试数据

```

任务清单

1. Skill IO 规范

- 每个 Skill 定义清晰的输入/输出 schema
- 示例——submission-tracker skill:

```

input:
  messages: MessageStream      # 来自 wechat-bridge
  challenges: ChallengeConfig  # 来自 config
output:
  submissions: SubmissionMatrix # 学生×挑战矩阵
  missing: [MissingItem]       # 缺失提交列表
  alerts: [Alert]              # 需要关注的异常

```

2. Property Graph 存储

- 选择存储后端 (任选) :
 - **SQLite + JSON**: 最轻量, 适合本地
 - **Neo4j / Memgraph**: 原生图数据库
 - **NetworkX + pickle**: 纯 Python, 适合原型
- 实现 CRUD 操作: 添加节点/边、查询关系、更新状态

3. Hook 集成（针对 OpenClaw）

- `on_message` hook → 触发 `msg-classifier` → 路由到相应 skill
- `on_schedule` hook → 定时触发 `class-report`
- `on_query` hook → 接收老师查询 → 搜索 graph → 返回结果

4. 自然语言查询接口

- 老师可以用自然语言问：
 - “张三提交了哪些挑战？”
 - “C4B 还有谁没交？”
 - “最近一周谁最活跃？”
 - “关于 GitHub 的常见问题有哪些？”
- 路由到对应 skill → 查询 graph → 格式化回答

交付物

- 完整的 `wechat-class-manager/` 目录结构
- 所有 `SKILL.md` 文件（符合标准格式）
- Property Graph 实现 + 测试
- 至少 3 个可演示的查询场景
- 姓名_C8_L3_AI日志.md

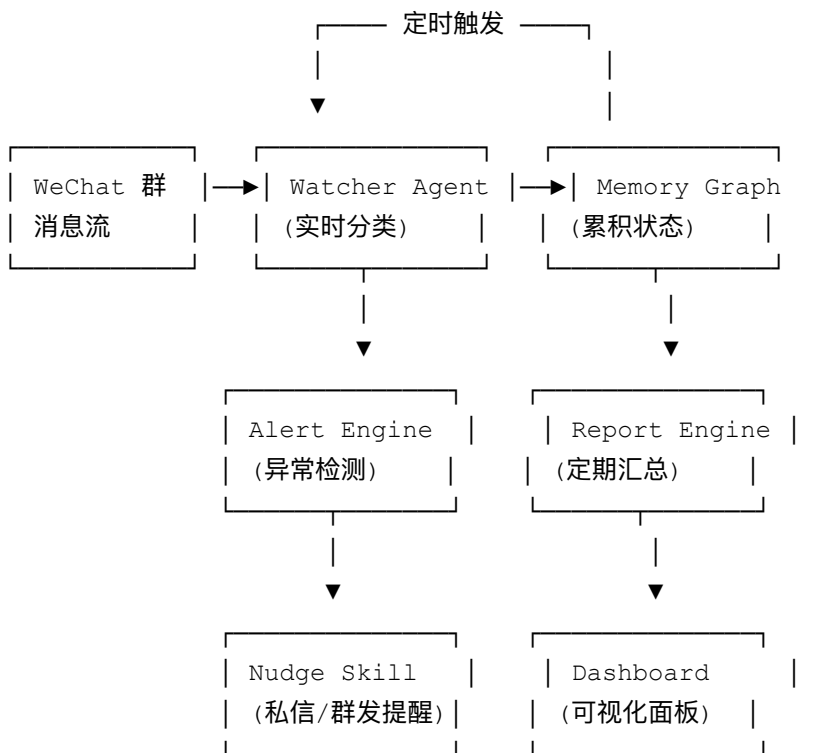
验收标准

- 每个 Skill 可被 `claude code` 或 `openclaw` 通过 `SKILL.md` 识别并调用
- Property Graph 能存储至少 20 个学生 × 8 个挑战的数据
- 自然语言查询能回答至少 5 种不同类型的问题
- 代码有测试，测试能通过

Level 4: 闭环系统——从被动查询到主动管理（Platinum 💎）

目标：Agent 不只是被动回答问题——它主动监控群动态、发出提醒、生成报告、检测异常。

从 Agent 到 Agent System



任务清单

1. Watcher Agent (群监控代理)

- 定期（如每小时/每天）拉取新消息
- 增量处理：只处理上次 checkpoint 之后的新消息
- 更新 Property Graph 状态

2. Alert Engine (预警引擎)

- 基于规则的异常检测：
 - 距 deadline 3 天，某学生仍未提交 → 发出提醒
 - 某学生连续 7 天零发言 → 标记为“沉默学生”
 - 某个挑战的提交率低于 30% → 老师预警
- 基于 KSTAR Delta 的学习信号：
 - 学生活跃度突然下降 (ΔE 偏差) → 主动关怀

3. Report Engine (报告引擎)

- 自动生成周报/月报：
 - 本周新增提交 N 件，涉及 M 个挑战
 - 最活跃学生 Top 5（发言量 + 质量）
 - 最佳 Q&A 对（被引用/被点赞最多）
 - 各挑战完成率柱状图
- 输出格式：Markdown → 可直接发群 / 转公众号

4. Dashboard（可视化面板）

- 本地 Web 仪表盘（承接 C6A）
- 实时展示班级状态
- 老师可以一键生成并发送报告

5. 双向通道（高阶选做）

- Agent 不只读群消息——还能 **写回** 群里
- 需要 Wechaty 或等价方案
- 场景：自动回复常见问题、催交提醒、公告确认

交付物

- 完整的闭环系统代码
- Watcher + Alert + Report 三个引擎可独立运行
- Dashboard 可视化面板（本地访问）
- 至少运行 1 周的真实数据演示
- 系统架构文档 + 部署指南
- 姓名_C8_L4_AI日志.md

验收标准

- Watcher 能增量处理新消息，不重复处理
- Alert 能在模拟场景中正确触发至少 3 种预警
- Report 能自动生成一份结构完整的周报
- Dashboard 能展示至少 5 个维度的班级数据

- 系统端到端可运行，有部署文档

与其他挑战的联动

联动挑战	关系
C3 群内参与	C8 自动追踪和量化 C3 的参与数据
C4 技能分享	C8 的每个模块本身就是一个可分享的 Skill
C5 GitHub	C8 代码必须托管在 GitHub
C6A 仪表盘	C8-L4 的 Dashboard 是 C6A 的升级版
C4C/C4D 公众号	C8 Report 可直接转为公众号文章
C3C 协作	C8 可以作为双人协作项目（一人做 Layer 1，一人做 Layer 3）

技术选型速查

Layer 1: WeChat 数据提取

工具	平台	特点	推荐度
<code>freestylefly/wechat-cli</code>	Linux/macOS	CLI-first, JSON 输出, AI-friendly	★★★★★
<code>sjzar/chatlog</code>	Win/macOS	HTTP API + TUI	★★★★
<code>afumu/wetrace</code>	Windows	取证级，多格式导出	★★★★
<code>JettChenT/wechat-exporter</code>	macOS	简单直接，一次性导出	★★★
自写脚本 (PyWxDump)	跨平台	灵活但需要更多工作	★★★

Layer 2: Agent Runtime

Runtime	优势	适合谁

Claude Code	最强 coding agent，直接调 SKILL.md	有 API 访问的同学
OpenClaw	本地运行，hook 系统，KSTAR 原生	了解 NEOLAF 架构的同学
Cursor	IDE 集成，写代码快	习惯 IDE 的同学
自写 Python agent	完全可控	想从零理解架构的同学

Layer 3: 存储

方案	复杂度	适合
SQLite + JSON columns	低	Bronze/Silver
SQLite + FTS5（全文搜索）	中	Gold
Neo4j / Memgraph	高	Platinum（原生图查询）
NetworkX → JSON export	低	快速原型

评分标准

维度	权重	说明
架构设计	25%	领域模型是否清晰？层次是否分明？Schema 是否合理？
功能完整性	25%	各 Level 的功能是否实现？是否可运行？
可复用性	20%	Skill 是否能被其他 Agent 调用？是否有标准接口？
代码质量	15%	结构、命名、测试、文档
AI 协作深度	15%	AI 日志是否展示了深度协作？是否有反思和迭代？

💡 AAR 反思模板（After Action Review）

每个 Level 完成后，写一份简短的 AAR：

AAR: C8 Level X

目标 vs 结果

- 计划做什么：
- 实际做到什么：
- 差距在哪里：

关键决策

- 做了哪些重要的技术选型决策？为什么？
- 如果重来，会做什么不同的选择？

困难与突破

- 遇到的最大困难是什么？
- 怎么解决的？（AI 帮了什么忙？自己想明白了什么？）

学到的技能

- 这个 Level 让我学会了哪些新技能？
- 这些技能可以迁移到什么场景？

KSTAR 编码（选做）

- K：我做这件事前已经知道什么？
- S：我面对的具体情境是什么？
- T：分解后的子任务是什么？
- A：我实际采取了什么行动？
- R：结果如何？与预期的差距？

起步建议

如果你是纯新手（从来没碰过 Agent/CLI）：

1. 先完成 C5A（GitHub 基础）
2. 用 Claude 帮你理解 wechat-cli 的 README
3. 从 `wechat_bridge.py` 开始，一个功能一个功能加
4. 目标：Bronze 🏅

如果你有编程经验：

1. 直接从 Level 2 开始设计分类器
2. 用 LLM API 做消息分类（比规则引擎更灵活）
3. 目标：Silver 🥈₂ → Gold 🥇₁

如果你是技术高手：

1. 从 Property Graph 领域模型开始设计

- 2. 直接封装为 Agent Skill
- 3. 挑战 Platinum  ——构建完整的闭环系统

与 KSTAR 理论的对应

对于想深入理解的同学，C8 的每个 Level 对应 KSTAR 认知循环的不同深度：

Level	KSTAR 对应	认知层次
Bronze	K→S：感知数据（看到消息流）	感知（Perception）
Silver	S→T：理解意图（分类消息）	理解（Comprehension）
Gold	T→A：生成行动（封装为 Skill）	应用（Application）
Platinum	A→R→K：闭环学习（监控+反馈+改进）	创造（Creation）

这不是巧合——好的系统设计本身就是一个认知过程。你在建造系统的过程中，也在训练自己的系统思维。

最终愿景：当这个系统运行起来，老师在任何时候都可以打开 Agent，问一句“这周班级状态如何？”，Agent 会基于微信群的真实数据，给出一份清晰、准确、可行动的回答。这不是科幻——这是你正在构建的现实。